

DIFFERENTIAL PRIVACY ON GRAPHS

Xiaoyang Wang



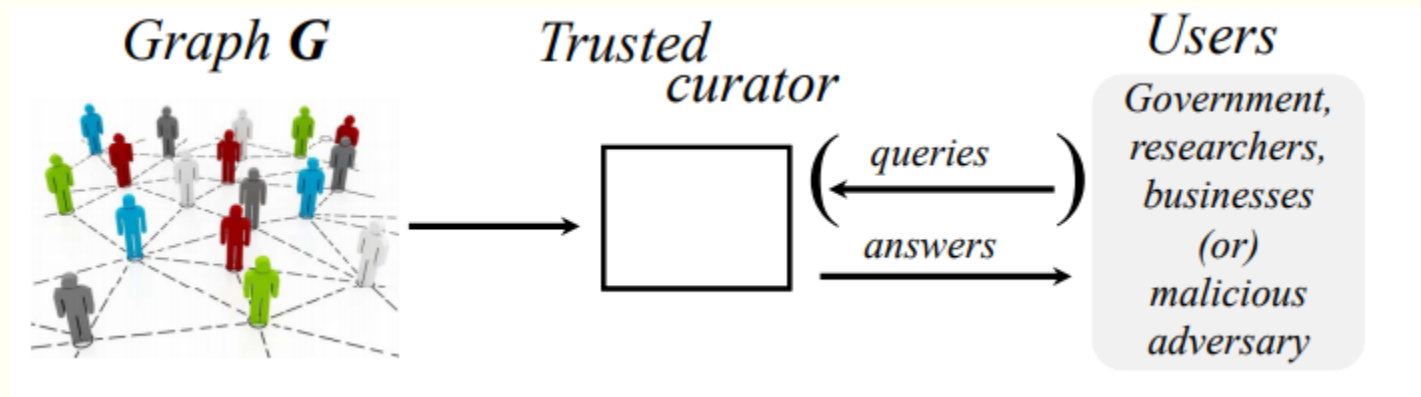
Graph Analysis

- Many datasets can be represented as graphs.[1]
 - “Friendships” in online social network
 - Financial transactions
 - Email communication
 - Romantic relationships
- Privacy is a big issue!

[1]<https://www.iacr.org/workshops/tcc2013/slides/8-3.pdf>

Analyzing Graphs with Node Differential Privacy – TCC 2013

- Goal:
 - Given a graph G
 - The users can send queries to a trusted curator.
 - The curator send answers back to users.
 - The users' information should not be revealed.
 - The answer should be relatively accurate.



Differential Privacy

- Goal: A change to one person's data will not significantly affect the output distribution of an analysis procedure.
 - Example:
 - Only one person in the world has disease A
 - Send a query to a hospital and ask whether they have patient with disease A
 - The answer to the query reveals the presence of the person in the hospital
- General method: Add noise.
- For graph data:
 - Node: Output doesn't reveal the inclusion or removal of a node and all its adjacent edge. (Picked)
 - Edge: Output doesn't reveal the inclusion or removal of an edge.
- Problem: Node-private algorithm can't provide a desirable accuracy.

Differential Privacy on Graph

- Goal:

- Find a node-private algorithm that is robust to the insertion of a new node in the graph.
 - In other words, the output doesn't change much after graph G is transformed to G'

- Challenge:

- The properties of a sparse graph can be altered dramatically by the insertion of a well-connected node.

- Example:

- Linear Queries

$$F_h(G) \stackrel{\text{def}}{=} \sum_{v \in G} h(\deg_v(G))$$

- $h(i) = i/2$, we have F_h counting the number of edges in a graph.
- In a sparse graph, the value of F_h will change a lot if we add or remove a node with high degree.

Differential Privacy - Preliminaries

- (epsilon)-differential privacy:

$$\Pr[\mathcal{A}(D_1) \in S] \leq e^\epsilon \times \Pr[\mathcal{A}(D_2) \in S]$$

- Epsilon is a positive real number
 - $\mathcal{A}$ is a randomized algorithm that takes a dataset as input
 - Dataset D_1 and D_2 differs on a single element
 - S is possible outputs
- Example:
 - Only one person in the world has disease A
 - D is the dataset contains the patient with disease A
 - D' is the modified dataset doesn't contain the patient with disease A
 - The $\Pr(D) = 1.0$, $\Pr(D') = 0.0$
 - Even infinitely large for epsilon is not enough

Differential Privacy – On Graphs

- (epsilon, delta)-differential privacy: $\Pr[\mathcal{A}(G) \in S] \leq \exp(\epsilon) \times \Pr[\mathcal{A}(G') \in S] + \delta$
- Global Sensitivity: $\Delta f = \max_{G, G' \text{ node neighbors}} \|f(G) - f(G')\|_1$
- Laplace Mechanism: $\mathcal{A}(G) = f(G) + \text{Lap}(\Delta f / \epsilon)^p$ (p is the dimension of output)
- Local Sensitivity: $LS_f(G) = \max_{G'} \|f(G) - f(G')\|$
- Smooth Bound:

<i>for all $G \in \mathcal{G}_n$:</i>	$S(G) \geq LS_f(G)$:
<i>for all neighbors $G, G' \in \mathcal{G}_n$:</i>	$S(G) \leq e^\beta S(G')$.
- Calibrating Noise to Smooth Bounds(based on Cauchy distribution):
 - Let S be a beta-smooth bound on LS
 - If $\beta \leq \epsilon / (\sqrt{2}p)$
 - We have epsilon-differentially private algorithm $\tilde{\mathcal{A}}(G) = f(G) + \text{Cauchy}(\sqrt{2}S(G)/\epsilon)^p$

Solutions

- Main idea: “Project” the graph onto a set of bounded-degree graphs.
 - On bounded-degree graph, node-private algorithm doesn’t suffer from the insertion of a well-connected node problem. In other words, the node-private algorithm is less sensitive.
 - With a carefully chosen degree threshold, the graph transformation loses relatively little information.
- Difficulty: The projection itself may be very sensitive.
- Overcome the difficulty with two techniques:
 - For a certain class of statistics, designed tailored projection operators using Lipschitz extensions
 - Analyze the “native” projection that simply discards high-degree nodes with efficient algorithms bounding the “local sensitivity” of this projection.

Lipschitz Extension

- Definition:

- f is the original function
- f' is a Lipschitz extension of f
- $\mathcal{G}$ is family of all graphs $\mathcal{G}$
- $\mathcal{G}_d$ is family of graphs $\mathcal{G}_d$ degree $\leq d$
- $\partial f'$ is sensitivity of f'
- $\partial_d f$ is sensitivity of f in

A function f' is a **Lipschitz extension** of f from $\mathcal{G}_d$ to $\mathcal{G}$ if

- f' agrees with f on $\mathcal{G}_d$ and
- $\partial f' = \partial_d f$

- For each function f , we can manually design a Lipschitz extension of it.
- The Lipschitz extension function has a lower sensitivity
- Then use it to approximate the original function in the projected space

Flow-based Lipschitz Extension

- The flow graph of G with parameter D , a source s and a sink t is a directed graph.
- The flow graph has capacitated edges:
 - Edges of capacity D from the source s to all nodes in V_l
 - Edges of capacity D from all nodes in V_r to the sink t
 - Unit-capacity edges (u_l, v_r) for all edges $\{u, v\}$ of G
- Two properties of this projection:
 - The global node sensitivity $\Delta v_{fl} \leq 2D$. $\partial f' = \partial_d f$
 - For all graphs G , the value $v_{fl}(G) \leq 2f_e(G)$, $v_{fl}(G)$ denotes the value of the maximum flow in the flow graph of G , $f_e(G)$ be the number of edges in G . Moreover, if G is D -bounded then $v_{fl}(G) = 2f_e(G)$.
 - The original function f is $f_e(G)$, which is counting the edge.
 - The sensitivity of it is D in $\mathcal{G}_d$

f' agrees with f on $\mathcal{G}_d$

Flow-based Lipschitz Extension

– Algorithm for Releasing the Number of Edges

■ Algorithm

Algorithm 1 ϵ -Node-Private Algorithm for Releasing $f_e(G)$

Input: parameters ϵ , D , n , and graph G on n nodes.

- 1: Let $\hat{e}_1 = f_e(G) + \text{Lap}(\frac{2n}{\epsilon})$ and threshold $\tau = \frac{n \ln n}{\epsilon}$.
 - 2: If $\hat{e}_1 \geq 3\tau$, **return** $\hat{e}_1$.
 - 3: Else compute the flow value $v_{\text{fl}}(G)$ given in Definition 4.1 with D .
 - 4: **return** $\hat{e}_2 = v_{\text{fl}}(G)/2 + \text{Lap}(\frac{2D}{\epsilon})$.
-

■ Lemma:

- If $f_e(G) \geq (5n \ln n)/\epsilon$, with probability at least $1 - 1/\ln(n)$, the output error is bounded $|\hat{e}_1 - f_e(G)| \leq (2n \ln \ln n)/\epsilon$

- If $f_e(G) < (n \ln n)/\epsilon$, and G satisfies α -decay for $\alpha > 1$, $D > \bar{d}$, $|\hat{e}_2 - f_e(G)| = O\left(\frac{2D \ln \ln n}{\epsilon} + \frac{\bar{d}^\alpha}{D^{\alpha-1}}\right)$

- 3τ in algorithm 1 seem an average of $(5n \ln n)/\epsilon$, and $(n \ln n)/\epsilon$, but the paper didn't elaborate more on this point.

Linear Programming-based Lipschitz Extension

- LP-based projection is used to release the number of a specified small template graph H in the input graph G . (sub-graph counts)

$$\begin{aligned} &\text{maximize} \quad \sum_{\text{copies } C \text{ of } H} x_C \text{ subject to:} \\ &\quad 0 \leq x_C \leq 1 \text{ for all variables } x_C \\ &\quad S_v \leq \Delta_D f_H \text{ for all nodes } v \in [n], \quad \text{where } S_v = \sum_{C: v \in V(C)} x_C. \end{aligned}$$

- Manually crafted another Lipschitz extension for subgraph matching in a similar way.

Generic Reduction Method

- What if we simply discard the nodes with large degrees via truncation?
 - The local sensitive can be computed via multiplying the sensitivity of query function and truncation function
 - Naïve truncation has high local sensitivity

$$S_{T_{naive}}(G) \leq \ell + 1/\beta + 1. \text{ (smooth upper bound on the local sensitivity)}$$

- ℓ is the number of nodes in degree range $D \pm \ln n / \beta$, β is the range
 - The bound depends on the number of nodes immediately around the cutoff D , may lead to large sensitivity

Generic Reduction Method

- Improvement:
 - Randomizing the cutoff- picks a random threshold in a range of bounded by a constant multiple of D
 - The smooth sensitivity of naive truncation is (likely to be) close to the average number of nodes of a random degree in the range - $nP_G(D)$
 - Saving a factor of roughly D in the introduced noise

$$\mathbb{E}_{\hat{D}}[S_{T_{naive}}(G)] \leq 3 \frac{nP_G(D)}{D} \cdot \frac{\ln n}{\beta} + \frac{1}{\beta} + 1$$

$P_G(D)$ is the fraction of nodes in G of degree greater than D

$\hat{D}$ is uniformly random
in the range $\{\check{D} + 1 + \ln n/\beta, \dots, 2\check{D} + \ln n/\beta\}$

Remark

- Proposed two general method:
 - Lipschitz extension
 - Generic reduction
- To use Lipschitz extension, design a Lipschitz extension function for each analytics task.
- Generic Reduction Method doesn't require manual effort but seemingly has a higher error

Generating Synthetic Decentralized Social Graph with Local Differential Privacy – CCS 2017

- As title said
- Current approach doesn't give enough accuracy due to
 - excessive noise injection
 - inability to retain informative graph structure

Straw-man Approach 1 – Randomized Neighbor List

- Given a social graph G , a set of users $U = \{u_1, \dots, u_n\}$, each user u in the set maintains a binary vector to represent local neighbor list.
- When sending the list to the data curator, each user perturb entries in their list with a probability $p = \frac{1}{1+e^\epsilon}$.
- Consider the Enron email graph (detailed in section 5), whose original density is 0.01%. Even with a small bit flipping probability $p = 0.01$, the expected density of the graph after neighbor list randomization would become 2%, an increase of 200 times

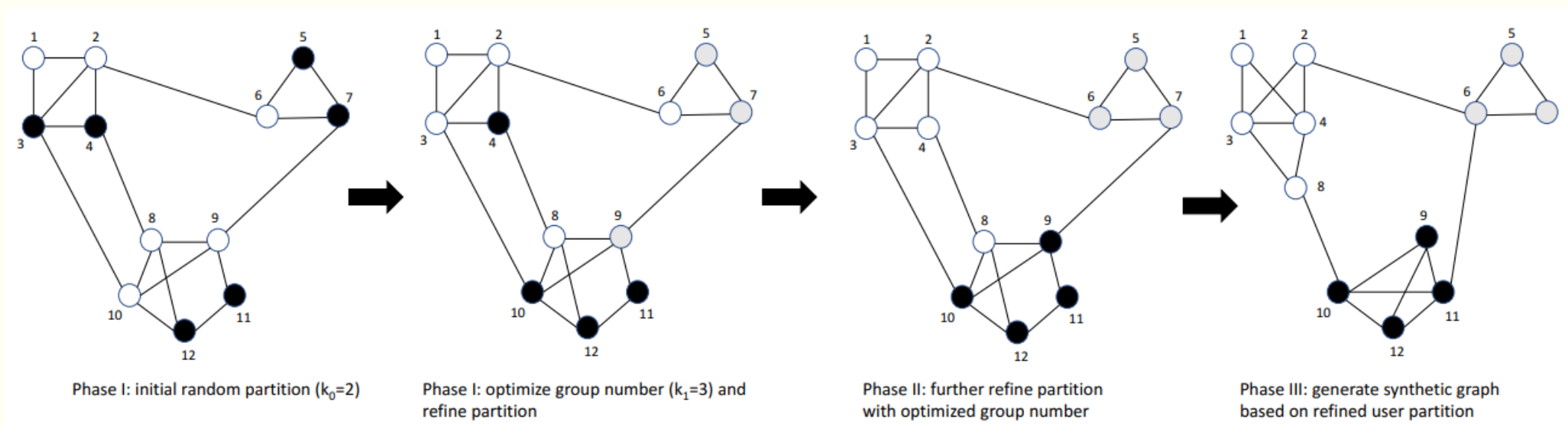
Straw-man Approach 2 – Degree-based Graph Generation

- DCG feeds graph statistics information such as node degrees into a social graph model to generate a graph.
- Using DGG, each user calculates his node degree, perturbs the degree under ϵ -edge differential privacy.
- For reasonably large values of ϵ , the perturbed degree is expected to be close to its true value, since the noise injected with the Laplace mechanism has a variance of $\frac{1}{\epsilon^2}$ for binary input

Observation from Straw-man Approaches

- RNL
 - Collects fine-grained information (i.e., neighbor lists)
 - Pays the price of heavy perturbations required to satisfy local differential privacy.
- DGG
 - Collects exclusively node degrees, it loses all other information of the underlying graph
 - Although DGG suffers less from perturbations noise

LDPGen - Overview



- Try to achieve a balance between noise(RNL) and information loss(DGG)
- Basic idea:
 - Partition the users into groups
 - Collect their degree vector(defined as number of neighbors)
 - Use collected degree vector to generate graph

LDPGen – Phase I: initial grouping

- Steps:
 - Partition users into k equal-sized groups
 - Compute degree vectors
 - Add Laplace noise to degree vectors
- To get a good partition, we need to decide the value of k and the partition method.
- Proposed an optimization-based approach to achieve better performance:
 - Objective function: $E[\frac{|\tilde{d}-d|}{d}]$ where d is the distance between neighbor vectors, $\tilde{d}$ is the distance between noisy degree vector

LDPGen – Phase I Noise analysis

- Objective function:

$$\begin{aligned} E \left[\frac{|\tilde{d} - d|}{d} \right] &= E \left[\frac{|\tilde{d} - \hat{d} + \hat{d} - d|}{d} \right] \\ &\leq \frac{1}{d} \left(E \left[|\tilde{d} - \hat{d}| \right] + E \left[|\hat{d} - d| \right] \right) \\ &= \frac{1}{d} \left(E \left[|\tilde{d} - \hat{d}| \right] + d - E \left[\hat{d} \right] \right) \end{aligned}$$

- Noise perturbation:

$$\begin{aligned} E \left[|\tilde{d} - \hat{d}| \right] &= E \left[\left| \sum_{i=1}^{k_1} |\delta_i^{\tilde{u}} - \delta_i^{\tilde{v}}| - \sum_{i=1}^{k_1} |\delta_i^{\hat{u}} - \delta_i^{\hat{v}}| \right| \right] \\ &\leq E \left[\sum_{i=1}^{k_1} |\delta_i^{\tilde{u}} - \delta_i^{\tilde{v}} - \delta_i^{\hat{u}} + \delta_i^{\hat{v}}| \right] \\ &\leq E \left[\left| \sum_{i=1}^{k_1} (\delta_i^{\tilde{u}} - \delta_i^{\hat{u}}) \right| \right] + E \left[\left| \sum_{i=1}^{k_1} (\delta_i^{\tilde{v}} - \delta_i^{\hat{v}}) \right| \right] \\ &= \sum_{i=1}^{k_1} \left(E \left[|\delta_i^{\tilde{u}} - \delta_i^{\hat{u}}| \right] + E \left[|\delta_i^{\tilde{v}} - \delta_i^{\hat{v}}| \right] \right) \end{aligned}$$

- Consider Laplace distribution:

$$E \left[\left| \delta_i^{\tilde{u}} - \delta_i^{\hat{u}} \right| \right] = 1/\epsilon_2$$

- Expected error:

$$E \left[|\tilde{d} - \hat{d}| \right] \leq k_1 \cdot 2\lambda = \frac{2k_1}{\epsilon_2}$$

Much smaller than RNL

LDPPGen – Phase I Optimal Group Number

- The number of partitions:

$$k_1 \approx \lceil \sum_{\eta=1}^{\eta_{max}} p_{\eta} \cdot k_1(\frac{1}{2}\eta) \rceil$$

- Weighted sum of optimal k_1 for each node based on their degree

$$k_1(d) = (d + \frac{d^2 - 2(1 + \sqrt{5})d + 1}{\epsilon_2})$$

- Optimal k_1 for node with degree d - L1 distances between their neighbor lists
- For nodes with the same degree η , we estimate their underlying L1 distance d heuristically

$$d = \frac{1}{2}\eta.$$

- The data curator runs the standard k-mean algorithm to cluster users into k_1 groups

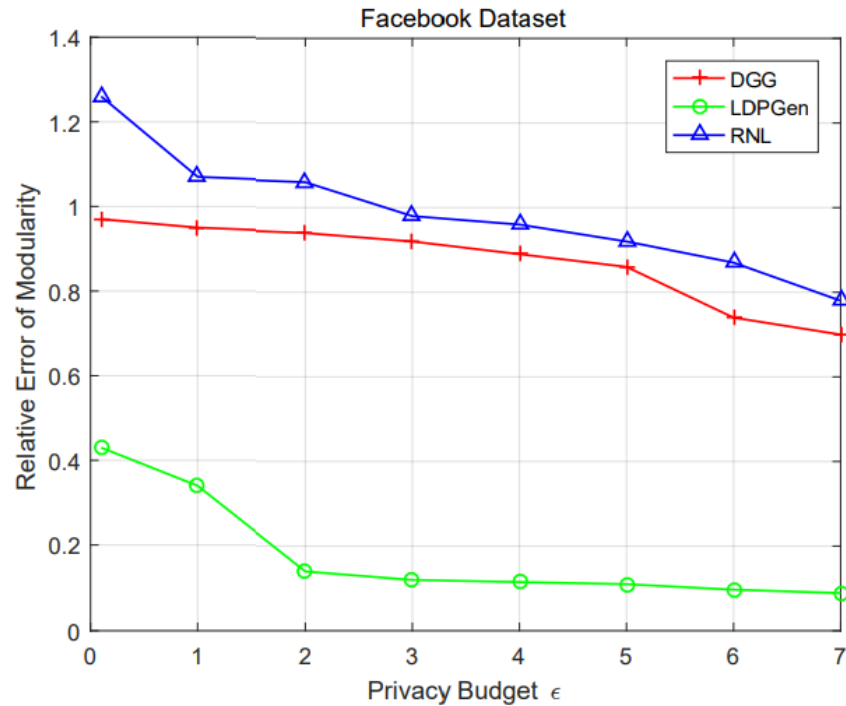
LDPGen – Phase II: grouping refinement.

- Performing k-mean clustering based on noisy degree vector at the beginning of Phase II.

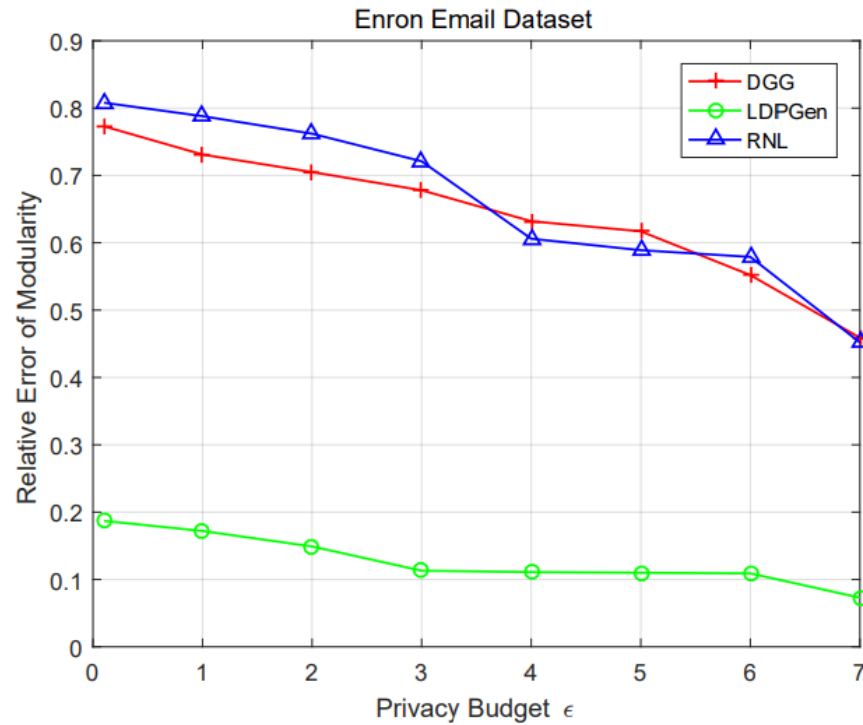
LDPGen – Phase III: graph generation

- Adapts BTER model to generate a synthetic social graph.

LDPGen – Experiment



(a) Facebook



(b) Enron

Figure 2: Effect of ϵ on modularity

Remark

- Proposed a three-phase generalized processing pipeline to generate synthetic graph from decentralized graphs while preserving privacy.
- The core of this algorithm is to find a good partition and an optimal group number.